

Reinforcement Learning Course

Deep Q-Learning Agent for Sailing Navigation

Project Report

Julien ROYER

June 9, 2026



Abstract

This report presents the design, implementation, and training of a Deep Q-Network (DQN) agent for autonomous sailing navigation in dynamic wind fields. The agent must navigate from a starting position to a goal location while optimally exploiting wind patterns. We describe our architectural choices, feature engineering approach, training methodology, and provide a detailed performance analysis.

1 Introduction

1.1 Problem Context

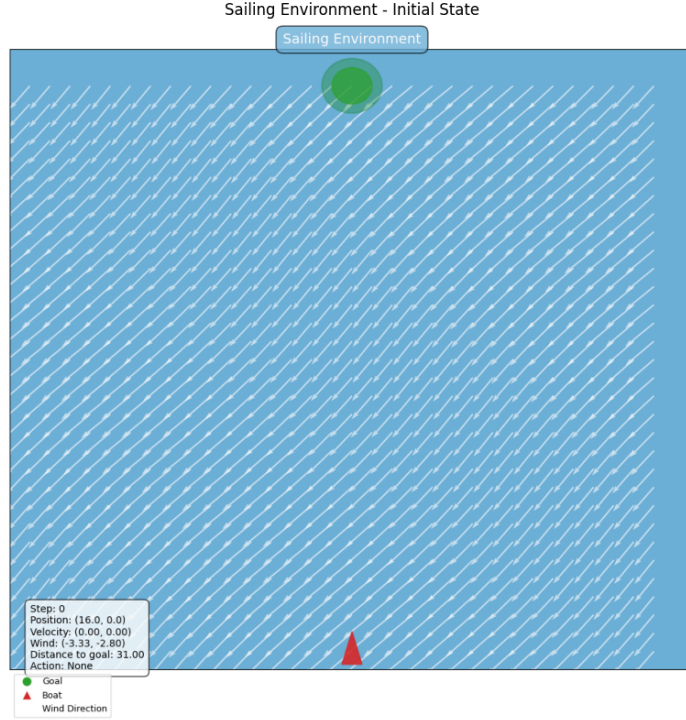


Figure 1: Sailing Environment Example

Our sailing navigation problem presents a challenge in reinforcement learning: an agent must learn to reach a goal position in a grid environment while dealing with spatially and temporally varying wind fields. The agent must exploit wind dynamics to reach the goal point as quickly as possible.

The environment is characterized by:

- **State space:** Position (x, y) , velocity (v_x, v_y) , local wind (w_x, w_y) , and a complete wind field.
- **Action space:** 9 discrete actions (8 directional movements + stay)
- **Dynamics:** Wind-assisted movement following sailing physics

2 Agent Architecture

2.1 Deep Q-Network Design

We employ a Deep Q-Network architecture to approximate the action-value function $Q(s, a)$. The network consists of:

- **Input layer:** 32-dimensional feature vector
- **Hidden layer:** 64 neurons with ReLU activation
- **Output layer:** 9 neurons (one per action)

The architecture can be expressed mathematically as:

$$h = \text{ReLU}(W_1 s + b_1) \quad (1)$$

$$Q(s, a) = W_2 h + b_2 \quad (2)$$

where $W_1 \in \mathbb{R}^{32 \times 64}$, $W_2 \in \mathbb{R}^{64 \times 9}$, and $s \in \mathbb{R}^{32}$ is the state feature vector.

2.2 Network Initialization

We use He initialization for weight matrices to facilitate gradient flow:

$$W_1 \sim \mathcal{N}\left(0, \sqrt{\frac{2}{32}}\right), \quad W_2 \sim \mathcal{N}\left(0, \sqrt{\frac{2}{64}}\right) \quad (3)$$

Biases are initialized to zero: $b_1 = 0_{64}$, $b_2 = 0_9$.

2.3 Feature Engineering

The raw observation space is extremely high-dimensional (2,054 values). We compress this into a 32-dimensional feature vector designed to capture essential navigation information:

2.3.1 Position and Goal Features (4 features)

- Normalized position: $x/32, y/32$
- Normalized goal distance: $\frac{\sqrt{(16-x)^2 + (30-y)^2}}{32}$
- Goal direction (cosine): $\cos(\text{atan2}(30 - y, 16 - x))$

2.3.2 Velocity Features (4 features)

- Normalized velocity components: $v_x/5, v_y/5$
- Velocity magnitude: $\sqrt{v_x^2 + v_y^2}/5$
- Velocity direction (cosine): $\cos(\text{atan2}(v_y, v_x))$

2.3.3 Local Wind Features (4 features)

- Normalized local wind: $w_x/5, w_y/5$
- Wind magnitude: $\sqrt{w_x^2 + w_y^2}/5$
- Wind direction (cosine): $\cos(\text{atan2}(w_y, w_x))$

2.3.4 Sailing Physics Features (4 features)

These capture the relationship between wind, velocity, and goal:

$$\text{Wind-velocity angle: } \cos(\theta_w - \theta_v), \sin(\theta_w - \theta_v) \quad (4)$$

$$\text{Wind-goal angle: } \cos(\theta_w - \theta_g), \sin(\theta_w - \theta_g) \quad (5)$$

where θ_w , θ_v , and θ_g are the wind, velocity, and goal angles respectively.

2.3.5 Regional Wind Analysis (16 features)

We divide the 32×32 grid into a 4×4 grid of regions (8×8 cells each) and compute average wind in each region:

$$\bar{w}_{\text{region}}^{(x,y)} = \frac{1}{|R|} \sum_{(i,j) \in R} w_{i,j}^{(x,y)} \quad (6)$$

We retain only the top 16 features (prioritizing regions along the path to goal) to limit dimensionality while maintaining spatial wind awareness.

2.4 Q-Value Computation

The action-value function is computed via a forward pass through the network:

Algorithm 1 Q-Value Computation

```

1: procedure COMPUTEQ( $s$ )
2:    $s \leftarrow \text{ExtractFeatures}(\text{observation})$  ▷ 32-dimensional vector
3:    $s \leftarrow \text{NanToNum}(s)$  ▷ Handle numerical issues
4:    $h \leftarrow \max(0, W_1 s + b_1)$  ▷ ReLU activation
5:    $h \leftarrow \text{clip}(h, 0, 50)$  ▷ Gradient stability
6:    $Q \leftarrow W_2 h + b_2$  ▷ Linear output
7:   return  $Q \in \mathbb{R}^9$ 
8: end procedure

```

The clipping operation at 50 prevents exploding activations during training while allowing sufficient dynamic range for learning.

3 Training Methodology

3.1 Deep Q-Learning with Experience Replay

Our training approach follows the DQN algorithm with several key components:

3.1.1 Target Network

We maintain two networks: a policy network $Q(s, a; \theta)$ and a target network $Q(s, a; \theta^-)$ with frozen weights. This stabilizes training by providing consistent targets during gradient updates.

The target for the TD update is computed as:

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-) \quad (7)$$

where $\gamma = 0.99$ is the discount factor.

3.1.2 Loss Function

We minimize the mean squared TD error:

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{i=1}^B (Q(s_i, a_i; \theta) - y_i)^2 \quad (8)$$

where $B = 64$ is the minibatch size.

3.1.3 Experience Replay Buffer

Transitions $(s, a, r, s', \text{done})$ are stored in a circular buffer of size 10,000. During training, we sample random minibatches to break temporal correlations and improve sample efficiency.

3.2 Gradient Computation

We then implement backpropagation :

$$\frac{\partial \mathcal{L}}{\partial W_2} = \frac{1}{B} H^T \delta \quad (9)$$

$$\frac{\partial \mathcal{L}}{\partial b_2} = \frac{1}{B} \sum_{i=1}^B \delta_i \quad (10)$$

$$\frac{\partial \mathcal{L}}{\partial W_1} = \frac{1}{B} S^T ((\delta W_2^T) \odot \mathbb{K}_{H>0}) \quad (11)$$

$$\frac{\partial \mathcal{L}}{\partial b_1} = \frac{1}{B} \sum_{i=1}^B ((\delta W_2^T) \odot \mathbb{K}_{H>0})_i \quad (12)$$

where H is the hidden layer activations, S is the state batch, δ is the TD error vector, and $\odot$ denotes element-wise multiplication.

3.3 Hyperparameters

Parameter	Value
Learning rate (α)	10^{-3}
Discount factor (γ)	0.99
Initial exploration (ϵ_0)	1.0
Final exploration ($\epsilon_{\min}$)	0.05
Exploration decay	Linear over 1000 episodes
Replay buffer size	10,000
Minibatch size	64
Target network update frequency	Every 5 episodes
Maximum episode length	1000 steps
Number of training episodes	1000

Table 1: Training hyperparameters

3.4 Training Procedure

3.4.1 Scenario Mixing

To promote generalization, we randomly sample from five training scenarios at each episode:

- `simple_static`
- `static_headwind`
- `training_1`
- `training_2`
- `training_3`

Each episode uses a random seed to ensure diversity within each scenario.

3.4.2 Exploration Strategy

We employ ϵ -greedy exploration with linear decay:

$$\epsilon_t = \max \left(0.05, 1 - \frac{t}{999} \right) \quad (13)$$

This balances exploration (high ϵ early) and exploitation (low ϵ later) throughout training.

4 Attempted Improvements and Challenges

4.1 Fine-tuning Attempts

Several approaches were explored to improve agent performance, though with limited success:

4.1.1 Shaped Rewards for Efficiency

Approach: We attempted to encourage faster navigation by adding intermediate rewards:

$$r_{\text{shaped}} = r_{\text{original}} - 0.1 \cdot (\text{steps}) \quad (14)$$

or:

$$r_{\text{shaped}} = r_{\text{original}} - 0.1 \cdot (\text{steps}) - 0.5 \cdot d(s, g) \quad (15)$$

where $d(s, g)$ is the distance to goal. Various step penalties were tested ($-0.01, -0.05, -0.1, -0.5$) to encourage efficiency without compromising goal-reaching behavior. No concrete improvement has been observed using this approach.

4.1.2 Curriculum Learning

Approach: We designed a curriculum starting with simpler scenarios:

1. Phase 1 (episodes 1-1000): Static uniform winds
2. Phase 2 (episodes 1001-2000): Dynamic scenarios
3. Phase 3 (episodes 2001-3000): All scenarios

Results: The agent learned quickly on both simple and dynamic scenarios but struggled to adapt when transitioning between them. The transition phases showed significant performance drops, suggesting forgetting.

5 Performance Analysis

5.1 Training Progress

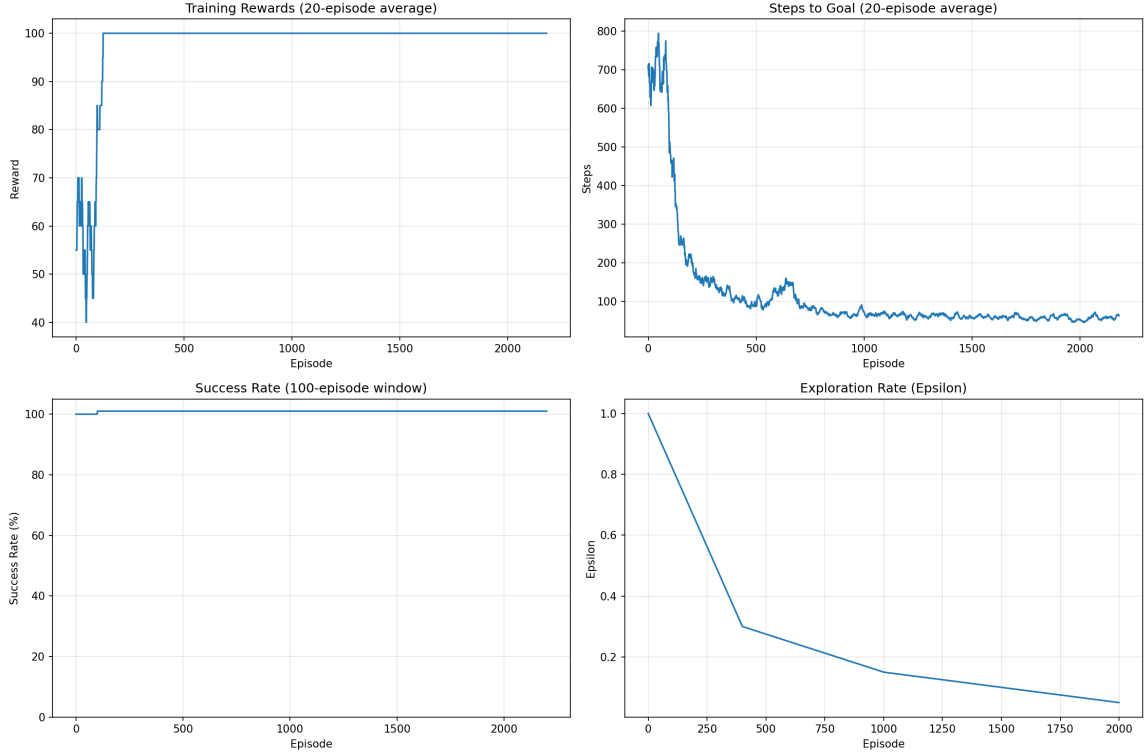


Figure 2: Example of a training run (3 phases, on only 2000 overall steps)

5.2 Training Scenarios Performance

Summary of Results Across All Wind Scenarios:

Wind Scenario	Success Rate	Mean Reward	Mean Steps
SIMPLE_STATIC	90%	31.38 ± 17.61	117.4 ± 46.1
STATIC_HEADWIND	100%	32.22 ± 26.39	117.8 ± 67.2
TRAINING_1	100%	59.15 ± 5.92	53.8 ± 10.8
TRAINING_2	100%	70.75 ± 4.24	35.6 ± 5.9
TRAINING_3	100%	51.73 ± 9.42	68.2 ± 17.9

Table 2: Performance across all wind scenarios

Scenario Set	Success Rate	Mean Reward	Mean Steps
Training Wind Scenarios	99%	49.05	78.6

Table 3: Average performance across training wind scenarios

5.3 Performance Distribution Analysis

A detailed analysis of episode lengths reveals a bimodal distribution:

- **Fast episodes (80%):** Completed in < 70 steps
- **Slow episodes (20%):** Require > 100 steps

This bimodal behavior significantly impacts the mean performance metric, as the slow episodes (often on simple scenarios) disproportionately increase average step counts.

5.4 Root Cause Analysis

The performance discrepancy between scenario types stems from fundamental differences in required navigation strategies:

5.4.1 Dynamic Scenarios (Strong Performance)

- Actively seeks wind corridors
- Adapts orientation based on conditions
- Executes smooth, efficient paths

5.4.2 Simple Scenarios (Degraded Performance)

- Inconsistent tacking angles
- May overshoot or undershoot optimal bearing
- Exhibits hesitation or oscillation in uniform conditions

5.4.3 Feature Representation Limitations

For uniform winds, our feature set becomes degenerate:

$$\text{Regional winds} \approx [w_0, w_0, \dots, w_0] \quad (\text{all similar}) \quad (16)$$

$$\text{Wind-goal angle} \approx \text{constant across episodes} \quad (17)$$

The agent must rely primarily on:

- Local wind direction (4 features)
- Wind-velocity angle (2 features)
- Goal distance and direction (3 features)

This reduced effective feature space (9 features instead of 32) may lack sufficient information for precise tacking control.

5.5 Future Directions

5.5.1 Hierarchical Policy

Implement a two-level architecture:

- **High-level:** Select strategy (exploit variations vs. disciplined tacking)
- **Low-level:** Execute chosen strategy

This explicitly models the two-strategy problem identified in our analysis.

5.5.2 Augmented Features

Add features that measure wind field properties:

$$\text{Spatial variance: } \sigma^2(\{w_i\}_{i=1}^N) \quad (18)$$

$$\text{Directional consistency: } \frac{1}{N} \sum_{i=1}^N \cos(\theta_{w_i} - \bar{\theta}_w) \quad (19)$$

These could help the agent recognize scenario type and adjust strategy accordingly.

6 Conclusion

We have developed and trained a Deep Q-Network agent for sailing navigation that achieves good performance on complex, dynamic wind scenarios but exhibits challenges on simpler uniform wind conditions. Despite these limitations, the agent demonstrates learning capabilities and successfully masters the complex task of wind-assisted navigation in dynamic environments. The insights gained from analyzing failure modes provide clear directions for future improvements, particularly the need for balanced training curricula and more sophisticated policy architectures.

7 Project Submission Folder

The submitted .zip archive contains the following files:

- `my_agent.py`: inference script with the trained weights hard-coded for evaluation.
- `dqn_agent.py`: implementation of the DQN agent class prior to training.
- `train_agent.py`: training script used to train the agent.